Data Mining (CS 145) Midterm Cheat Sheet by Patricia Xiao

Models

Model	Data Type	Task Type
Linear Regression	Vector	Prediction
Logistic Regression	Vector	Classification
Decision Tree	Vector	Classification
SVM	Vector	Classification
NN	Vector	Classification
KNN	Vector	Classification
K-means	Vector	Clustering
hierarchical clustering	Vector	Clustering
DBSCAN	Vector	Clustering
Mixture Models	Vector	Clustering

Basic Concepts

- **Dispersion:** Quartiles& Inter Range (Q_1 25%, Q_3 75%, IQR $= Q_3 - Q_1$, Outlier 1.5 IQR away $Q_{1/3}$), 5 n Summary: min, Q_1, median, Q_3, max
- Bias: $E(\hat{f}(x)) - f(x)$, Variance: $Var(\hat{f}(x)) = E[(\hat{f}(x) - E(\hat{f}(x)))^2]$, $E[(\hat{f}(x) - f(x) - \epsilon] = bias^2 + variance + noise$; $E(\epsilon) = 0$, $Var(\epsilon) = \sigma^2$; bias→underfit; variance→overfit
- Model Evaluation and Selection: K-way cross validation, AIC ($2k - 2\ln(\hat{L})$) & BIC ($k\ln(n) - 2\ln(\hat{L})$) (k params, n objs), Stepwise feature selection (forward: add, backward: from full model)
- Generalized Linear Model (GLM): exponential family, $p(y;\eta) = b(y)exp(\eta^T T(y) - a(\eta))$; **linear decision boundary**
- **Bagging:** Bootstrap Aggregating (multi-datasets → multiple classifiers → combine classifiers)
- **Kernel:** $K(x_i, x_j) = \Phi(x_i)^T \Phi(x_j)$
- Chain rule: $\partial J/\partial x = (\partial J/\partial y)(\partial y/\partial x)$
- Minkowski distance (l_h): $d(x,y) = \sqrt[h]{\sum_i^d (x_i - y_i)^h}$, l_1 Manhattan, l_2 Euclidean, l_∞ supremum; triangle inequality applies ($d(i,j) \le d(i,k) + d(k,j)$).
- **Confusion Matrix:** True / False for correctness, Positive / Negative for result
- Multi-class classification: All-vs-all (AVA) is better than One-vs-All (OVA)

Formula

1. $\sigma^2 = E[(X - E(X))^2] = E(X^2) - E^2(X)$
2. $\|\alpha\|^2 = \alpha^T \alpha$ where α is a vector.
3. $(AB)^T = B^T A^T$, $(AB)^{-1} = B^{-1}A^{-1}$
4. $\frac{\partial(Ax)}{\partial x} = A$, $\frac{\partial(AX)}{\partial X} = A^T$
5. $\frac{\partial(x^T Ax)}{\partial x} = x^T(A + A^T)$, $\frac{\partial(X^T A^T AX)}{\partial X} = 2A^T AX$
6. $X \sim N(\mu, \sigma^2) \Rightarrow f(X = x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
7. $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
8. $\log(a^b) = b\log a$, $\log(ab) = \log(a) + \log(b)$
9. Classifiers $f_i(x)$: $var(\frac{\sum_i f_i(x)}{t}) = var(f_i(x))/t$
10. $a \cdot b = \sum a_i b_i = \|a\|\|b\| \cos(a,b)$
11. normal $\mathbf{n}$, any vector in plane, $\mathbf{x}$, $\mathbf{n} \cdot \mathbf{x} = 0$
12. covariance: $\sigma(X_1, X_2) = E(X_1 - \mu_1)(X_2 - \mu_2)$

Tools

1. **mean - mode = 3 × (mean - median)** mode(peak)∼median∼mean(∼ tail)
2. Z-score (normalization): $Z = \frac{x-\mu}{\delta}$ (robust: mean absolute deviation, $z_{jf} = \frac{x_{if} - mean_f}{sum_f}$) (nominal: dummy variable(s), ordinal: $(r-1)/(M-1)$) where r, M start from 1.
3. **Logistic / Sigmoid** Function: $\sigma(x) = \frac{1}{1+e^{-x}}$
4. Entropy: $H(Y) = -\sum_{i=1}^m p_i \log(p_i)$; Conditional Entropy: $H(Y|X) = \sum_x p(x) H(Y|X = x)$
5. Cross Entropy Loss: $H(q,p) = -\sum_k q_k \log(p_k)$
6. Lagrange multiplier α is used to solve Quadratic Programming (e.g. SVM)
7. Soft margin (allow moving at a cost): minimizing $\Phi(w) = 1/2w^T w \Rightarrow \Phi(w) = 1/2w^T w + C\sum \zeta_i$, limitation $y(w^T x_i + b) \ge 1 \Rightarrow y(w^T x_i + b) \ge 1 - \zeta_i$ ($\zeta_i \ge 0$); doesn't affect the solution of SVM.
8. ROC (Receiver Operating Characteristics): TP rate (y-axis) - FP rate (x-axis), score = area below curve
9. **Dendrogram**: the hierarchical, cut to clusters.

Linear Regression

$y = x^T\beta$ where bias term $x_{i0} = 1$, x: $(n \times (p+1))$ matrix, y: $(n \times 1)$ vector, β: $((p+1) \times 1)$ vector. **Continuous** $y = x\beta^T$. (**OLS**, Ordinary Least Square) $J(\beta) = \frac{1}{2n}(X\beta - y)^T(X\beta - y) = \frac{1}{2n}(\beta^T X^T X\beta - y^T X\beta - \beta^T X^T y + y^T y)$.
Closed form solution: $\frac{\partial J}{\partial \beta} = 0$, $\hat{\beta} = (X^T X)^{-1} X^T y$
Gradient Descent: $\beta^{(t+1)} := \beta^{(t)} - \eta\Delta$
Batch GD: (converge) $\Delta = \frac{\partial J}{\partial \beta} = \sum_i x_i(x_i^T\beta - y_i)/n$
Stochastic GD: (n times) $\Delta = -(y_i - x_i^T\beta^{(t)})x_i$
LR with Probabilistic Interpretation: (using **MLE**, *Maximum Livelihood Estimation*) $L(\beta) = \prod_i p(y_i|x_i,\beta) = \prod_i p(N(x_i^T\beta, \sigma^2)) = \prod_i \frac{1}{\sqrt{2\pi\sigma^2}} exp\{-\frac{(t_i - x_i^T\beta)^2}{2\sigma^2}\}$
Invertible $X^T X$: add $\lambda\sum_{j=1}^p \beta_j^2$ to $\sum_i (y_i - x_i^T\beta)^2$ (**Ridge Regression**, or linear regression with l_2 norm)
Non-linear Correlation: create new terms e.g. x^2

Logistic Regression

Generalized **linear** model (GLM).
$P(Y = 1|X,\beta) = \sigma(X^T\beta) = \frac{e^{X^T\beta}}{1+e^{X^T\beta}}$
$P(Y = 0|X,\beta) = 1 - \sigma(X^T\beta) = \frac{1}{1+e^{X^T\beta}}$
$Y|X,\beta \sim Bernoulli(\sigma(X^T\beta))$
MLE: $L = \prod_i p_i^{y_i}(1-p_i)^{1-y_i}$, p_i is $P(Y = 1|X,\beta)$
Eq to max log likelihood $L = \sum_i (y_i x_i\beta - \log(1 + e^{x_i^T\beta}))$
Gradient ascent $\beta^{new} = \beta^{old} + \eta\frac{\partial L(\beta)}{\partial\beta}$
Newton-Raphson update $\beta^{new} = \beta^{old} - (\frac{\partial^2 L(\beta)}{\partial\beta})^{-1}\frac{\partial L(\beta)}{\partial\beta\partial\beta^T}$
Cross Entropy Loss (p for prediction, q for ground truth, $(q_0, q_1)|_{y=0} = (1,0)$, $(q_0,q_1)|_{y=1} = (0,1)$, $(p_0,p_1) = (P(Y=0), P(Y=1))$: $H(p,q) = -yx^T\beta + log(1 + e^{x^T\beta})$

EM Algorithm

A framework to approach maximum likelihood.
$p(x_i, z_i = C_j) = w_j f_j(x_i)$, $p(x_i) = \sum_j w_j f_j(x_i)$
$p(D) = \prod_i p(x_i) = \prod_i \sum_j w_j f_j(x_i)$
$\log(p(D)) = \sum_i \log(\sum_j w_j f_j(x_i))$
E(expectation)-step assigns objects to clusters.

$$w_{ij}^{t+1} = p(z_i = j|\theta_j^t, x_i)$$
$$\propto p(x_i|z_i = j, \theta_j^t)p(z_i = j) = f_j(x_i)w_j$$

M(maximization)-step finds the new clustering w.r.t. conditional distribution $p(z_i = j|\theta_j^t, x_i)$.

$$\theta^{t+1} = \underset{\theta}{\operatorname{argmax}} \sum_i \sum_j w_{ij}^{t+1} \log L(x_i, z_i = j|\theta)$$

Decision Tree

m for $|y|$ in D, v for $|A|$
Expected Information needed to classify a tuple in D: $Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$
Info after split A: $Info_A(D) = \sum_{j=1}^{v} \frac{D_j}{D} \times Info(D_j)$
Info Gain (ID3): $Gain(A) = Info(D) - Info_A(D)$
Info gain biases towards multivalued attributes.
$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{D_j}{D} \times \log_2(\frac{D_j}{D})$
GR (C4.5): $GainRatio(A) = Gain(A)/SplitInfo(A)$
GR biases towards unbalanced splits.
$Gini(D) = 1 - \sum_{j=1}^{m} p_j^2$ for impurity
$Gini_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} Gini(D_j)$
Gini (CART): $\Delta Gini(A) = Gini(D) - Gini_A(D)$
Gini index also biases towards multivalued attributes.
STOP: same class; last attr; no sample (maj. vot.)
Avoid Over Fitting: Pre/Post-pruning, random forest
Classification $\rightarrow$ **Prediction:** Maj. Vote $\rightarrow$ e.g. Avg for leaf node.
turn to regression tree, $Var(D_j) = \sum_{y \in D_j} (y - \overline{y})^2 / |D_j|$, look for the lowest **weighted average variance** $Var_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Var(D_j)$
A different view: leaf = box in the plane
Random forest is a set of trees, ensemble, bagging, good at **classification**, handles large & missing data, not good at **predictions**, lack interpretation.

SVM

$y = sign(\mathbf{W} \cdot \mathbf{X} + b)$, separating hyperplane $y = 0$
SVM searches for **M**aximum **M**arginal **H**yperplane
To Maximize Margin $\rho = \frac{2}{\|\mathbf{w}\|}$, w. Lagrange multiplier α, $L(w, b, \alpha) = \frac{1}{2} w^T w - \sum_{i=1}^{N} \alpha_i (y_i (w^T x_i + b) - 1)$.
$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{N} \alpha_i y_i x_i = 0$, $\frac{\partial L}{\partial b} = -\sum_{i=1}^{N} \alpha_i y_i = 0$
Solution: $w = \sum \alpha_i y_i x_i$, $b = y_k - w^T x_k$
$f(x) = w^T x + b = \sum \alpha_i y_i x_i^T x + b$ default threshold 0
Linear v.s. Non-linear SVM: **Kernel**
Non-linear Decision Boundary: $f(x) = w^T \Phi(x) + b = \sum \alpha_i y_i K(x_i, x) + b$
Scalability: CF-Tree, Hierarchical Micro-cluster, selective declustering (decluster the clusters who could be support cluster; support cluster: centroid on support vector)

Perceptron (Single Unit)

$$x_i \xrightarrow{w_i} \sum(+b) \xrightarrow{f} o$$

Input vector x, Weight vector w, Bias b, weighted sum, going through activation function f, reach output o.

Backpropagation (BP)

Stochastic GD + Chain Rule
Special case: Sigmoid + Square loss, 2 layers
Assume: i, j, k are input, hidden, output layers' denotion, and O for output, T for true value.
$Err_k = O_k(1 - O_k)(T_k - O_k)$, $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$, $w_{ij} = w_{ij} + \eta Err_j O_i$ and $w_{jk} = w_{jk} + \eta Err_k O_j$, $\theta_j = \theta_j + \eta Err_j$ and $\theta_k = \theta_k + \eta Err_k$.
$\frac{\partial J}{\partial w_{ij}} = \frac{\partial J}{\partial O_k} \frac{\partial O_k}{\partial O_j} \frac{\partial O_j}{\partial w_{ij}} = -\sum_k [(T_k - O_k)][O_k(1 - O_k) w_{jk}][O_j(1 - O_j) O_i]$

Neural Network (NN)

$n_{layers} = n_{hidden} + n_{output}(1)$
Feed-forward, Non-linear regression, capable of any continuous function.
Backpropagation is used for learning.

k - Nearest Neighbors (kNN)

Lazy learning (instead of eager), instance-based
Consider k nearest neighbors; maj. voting or average. (Could be distance-weighted.)
Curse of dimensionality: influence of noise
Get rid of irrelevant features; select proper k.
Proximity refers to similarity or **dissimilarity**. Always applies to binary values. If nominal, could do simple matching, or use a series of binary to represent a non-binary; ordinal: rank, normalize $z_{if} = \frac{r_{if} - 1}{M_f - 1}$.
Proximity could be measured by $\frac{|(0,1)| + |(1,0)|}{all}$ for symmetric variables, $\frac{|(0,1)| + |(1,0)|}{all - |(0,0)|}$ or Jaccard coefficient (similarity) $\frac{|(1,1)|}{all - |(0,0)|}$ for asymmetric.
Mixed type attributes: weighted combine.
Another method: cosine similarity $cos(d_1, d_2)$

Evaluation: Classification

Holdout method; Cross-validation (k-fold) LOO.
Confusion Matrix: True / False Positive / Negative
Accuracy = (TP + TN) / All
Error Rate = (FP + FN) / All
Sensitivity = TP / P (P = TP + FN)
Specificity = TN / N (N = FP + TN)
Precision = TP / P' (P' = TP + FP)
Recall = TP / P = Sensitivity
F_1 / F-score = $\frac{2 \times Precision \times Recall}{Precision + Recall}$
$F_\beta = \frac{(1+\beta^2) \times Precision \times Recall}{\beta^2 \times Precision + Recall}$ (R: P = $\beta : 1$)
ROC curve: TP rate (y) - FP rate (x). (area under)
TPR = TP / P, FPR = FP / N

Clustering

K-means: $J = \sum_{j=1}^{k} \sum_i w_{ij} \|x_i - c_j\|^2$
Assign $w_{ij} = 1$ to each x_i closest c_j; assign the center to be new centroid; stop when no change. O(tkn). For continuous, convex-shaped data, sensitive to noise.
K-modes: $mean \rightarrow mode$, for categorical data
K-medoids: representative objects, e.g. PAM (s)
Hierarchical: bottom-up **Agglomerative Nesting (AGNES)** merges two closest clusters until end up in 1; top-down **DIANA (Divisive Analysis)**. $O(n^2)$.
Cluster Distance: Single link for min element-wise dist; Complete link for max; average for avg element pairs dist; centroid, medoid (center obj).
DBSCAN: Set Eps ϵ and $MinPts$. Neighborhood defined as $N_\epsilon(q) : \{p \in D | dist(p, q) \leq \epsilon\}$. Core point $|N_\epsilon(q)| \geq MinPts$. p is **directly density-reachable** from q if q is core point and $p \in N_\epsilon(q)$; **density-reachable** if $q \rightarrow p_2 \rightarrow \cdots \rightarrow p$; **density-connected** if $o \rightarrow \cdots \rightarrow p \bigwedge o \rightarrow \cdots \rightarrow q$. Cluster: max set density-connected points. Individual points are **noise**. DFS $O(n \log n)$ w. spacial index, else $O(n^2)$.
Mixture Model: soft clustering ($w_{ij} \in [0, 1]$ rather than $w_{ij} \in \{0, 1\}$), joint prob of object i and cluster C_j: $p(x_i, z_i = C_j) = w_j f_j(x_i)$, using EM algorithm.
Gaussian Mixture Model (GMM): $\supset$ **k-means**
Generative model, for each object, pick cluster Z, from $X|Z \sim N(\mu_Z, \sigma_Z^2)$ sample value; Overall likelihood function $L(D|\theta) = \prod_i \sum_j w_j p(x_i | \mu_j, \sigma_j^2)$;
E $w_{ij}^{t+1} = (w_j^t p(x_i | \mu_j^t, (\sigma_j^2)^t)) / (\sum_k w_k^t p(x_i | \mu_k^t, (\sigma_k^2)^t))$,
M $\mu_j^{t+1} = (\sum_i w_{ij}^{t+1} x_i) / (\sum_i w_{ij}^{t+1})$, $(\sigma_j^2)^{t+1} = (\sum_i w_{ij}^{t+1} (x_i - \mu_j^{t+1})^2 / (\sum_i w_{ij}^{t+1})$, $w_j^{t+1} = \sum_i w_{ij}^{t+1} / n$ (in 1-d case)
Why EM works? E-Step find **tight** lower bound L of ℓ at θ_{old}, M-Step find θ_{new} to maximize the lower bound. $\ell(\theta_{new}) \geq L(\theta_{new}) \geq L(\theta_{old}) = \ell(\theta_{old})$

Evaluation: Clustering

extrinsic (supervised) vs. intrinsic (unsupervised)
$purity(C, \Omega) = \frac{1}{N} \sum_K \max_j |c_k \cap \omega_j|$ (C out, Ω truth)
Normalized Mutual Information:
$NMI(C, \Omega) = \frac{I(C, \Omega)}{\sqrt{H(C) H(\Omega)}}$
$I(C, \Omega) = \sum_k \sum_j P(c_k \cap \omega_j) \log \frac{P(c_k \cap \omega_j)}{P(c_k) P(\omega_j)} = \sum_k \sum_j \frac{|c_k \cap \omega_j|}{N} \log \frac{N |c_k \cap \omega_j|}{|c_k| |\omega_j|}$
$H(\Omega) = -\sum_j P(\omega_j) \log P(\omega_j) = -\sum_j \frac{|\omega_j|}{N} \log \frac{|\omega_j|}{N}$
Precision and Recall: same / different class / cluster
Select k: plot square loss - k, larger k smaller cost, find **knee** points; BIC penaltize; Cross validation